Consola para Administración de Máquinas Virtuales GNU/Linux

Jorge Acosta Andrey Tombe

Universidad del Quindío
Facultad de Ingeniería
Ingeniería de Sistemas y Computación
Computación en la Nube
Octubre del 2026

***Resumen*—(Redactar al final del informe.)**

***Index Terms*—palabra clave, palabra clave, palabra clave**

I. Introducción

La administración de sistemas operativos GNU/Linux es una actividad fundamental en el área de computación en la nube. Tareas como la instalación de paquetes, la configuración del servidor SSH, la gestión de servicios y el balanceo de carga se realizan, en muchos entornos, de manera manual y a través de comandos introducidos directamente en una terminal. Este enfoque, si bien es válido, resulta propenso a errores de digitación, difícil de estandarizar y poco accesible para usuarios que no dominan la línea de comandos.

A lo largo del curso de Computación en la Nube se han ejecutado estas operaciones de forma tradicional. En el presente trabajo se propone convertir dichas tareas en operaciones que pueden solicitarse desde una aplicación web desarrollada en el lenguaje Go. La aplicación funciona como una consola de automatización para una máquina virtual que ejecuta GNU/Linux Debian 13, integrando las operaciones del sistema en una interfaz de usuario que permite validar las entradas, ejecutar las acciones de manera no interactiva y reportar el resultado de cada proceso, incluyendo mensajes de error comprensibles y la información técnica necesaria para el diagnóstico.

II. Marco Teórico

II-A. Máquinas virtuales y virtualización

La virtualización permite ejecutar múltiples sistemas operativos aislados sobre un mismo hardware físico mediante un hipervisor, que intermedia el acceso a los recursos de la máquina. En este trabajo se utiliza Oracle VM VirtualBox, un hipervisor de tipo 2 (alojado sobre un sistema operativo anfitrión) multiplataforma, disponible para GNU/Linux y Windows [1].

VirtualBox se administra tanto desde su interfaz gráfica como desde la utilidad de línea de comandos `VBoxManage`, incluida en toda instalación y disponible de forma idéntica en todos los sistemas operativos soportados [1]. Esta característica es relevante para la consola: cualquier operación que la aplicación deba realizar sobre la máquina virtual (consultar su estado, obtener su dirección IP, ejecutar un programa dentro de ella) puede expresarse como llamadas a `VBoxManage`, lo que la convierte en la base de la comunicación inicial con la máquina virtual.

Las Guest Additions son controladores y servicios que se instalan dentro de la máquina virtual y mejoran la integración con el anfitrión: redimensionado de pantalla, portapapeles compartido, directorios compartidos y, en particular, la publicación de propiedades del sistema invitado [2]. Entre estas propiedades se encuentra la dirección IP de la máquina, publicada como una propiedad consultable desde el anfitrión con el nombre `/VirtualBox/GuestInfo/Net/0/V4/IP`. Este mecanismo resuelve un problema práctico del proyecto: antes de que exista cualquier servicio de red configurado dentro de la máquina (como SSH), la consola necesita conocer su dirección para poder establecer la primera conexión.

II-B. Sistema operativo GNU/Linux Debian

II-C. Gestión de paquetes con APT

II-D. Protocolo y servicio SSH

El protocolo Secure Shell (SSH) proporciona canales seguros sobre redes no confiables, con autenticación cifrada, confidencialidad e integridad de las comunicaciones [3]. En servidores GNU/Linux el servidor de referencia es OpenSSH, que implementa el servicio `sshd` y el cliente homónimo.

SSH admite autenticación por contraseña y por par de llaves criptográficas asimétricas. En el esquema por llaves, el cliente posee una llave privada y el servidor guarda la llave pública correspondiente en el archivo `~/.ssh/authorized_keys` de cada usuario; el servidor verifica la firma del cliente sin transmitir secretos, y es posible deshabilitar por completo el acceso por contraseña [3]. Este es el mecanismo que la consola configura

en la máquina virtual: una vez instalado y habilitado SSH, toda la comunicación posterior entre la consola y la máquina se realiza por este canal, reemplazando la comunicación inicial basada en Guest Additions.

Además de la ejecución interactiva de comandos, el cliente OpenSSH permite ejecutar una orden remota de forma no interactiva (`ssh usuario@host orden`) y transferir archivos mediante `scp`, lo que lo convierte en una interfaz de programación completa para automatizar administración de sistemas [4].

II-E. Gestión de servicios con systemd

II-F. Balanceo de carga con HAProxy

II-G. Lenguaje de programación Go y aplicaciones web

Go es un lenguaje compilado, de tipado estático y concurrencia ligera, diseñado para producir binarios simples y multiplataforma [4]. Compilar el mismo código para GNU/Linux y Windows sin dependencias en tiempo de ejecución encaja con el objetivo de esta consola, ya que la aplicación debe poder ejecutarse en el equipo anfitrión independientemente de su sistema operativo.

La interfaz web se sirve con el paquete estándar `net/http` y sigue una arquitectura cliente-servidor: el navegador presenta los módulos de administración y envía formularios; el servidor en Go valida las entradas, ejecuta las operaciones sobre la máquina virtual y devuelve el resultado.

Para ejecutar operaciones externas, Go ofrece el paquete `os/exec`, que crea y controla procesos del sistema operativo capturando su salida, su error y su código de salida [4]. Este es el primitivo sobre el que se construye la biblioteca de conectores: la consola no invoca `vboxmanage` ni `ssh` directamente desde su lógica, sino a través de procesos externos lanzados de forma controlada, lo que permite usar los mismos binarios en Windows y en GNU/Linux.

II-H. Arquitectura de puertos y adaptadores

Para mantener el núcleo de la aplicación independiente de la infraestructura concreta se aplica el patrón de arquitectura de puertos y adaptadores (también llamado arquitectura hexagonal) [5]. En este patrón, el núcleo define *puertos*: interfaces que expresan lo que la aplicación necesita del exterior (ejecutar un programa, abrir una sesión con la máquina, conocer su dirección). Los *adaptadores* son implementaciones concretas de esos puertos, que traducen el lenguaje de una tecnología específica al lenguaje del dominio.

En este proyecto los puertos son `Executor` (ejecutar un proceso en el anfitrión), `Machine` (obtener dirección y estado de la máquina virtual), `Connector` (establecer sesiones) y `Session` (ejecutar órdenes y transferir archivos dentro de la máquina). Los adaptadores implementados son `localproc` (procesos del sistema operativo), `vbox` (VirtualBox mediante `vboxmanage` y Guest Additions) y `ssh` (cliente OpenSSH). De este modo, el núcleo no sabe que se usa Guest Additions ni VirtualBox: solo conoce interfaces, y cambiar una tecnología por otra (u otra distinta) no afecta a la lógica de la consola.

III. Diseño de la Solución

III-A. Escenario de Aplicación

III-A1. Descripción del Escenario: La solución se desarrolla y se prueba sobre el escenario de la fig. 1. El *equipo host* es el equipo físico donde se instala Oracle VM VirtualBox 7 y se ejecuta la consola `vmcons`; su sistema operativo puede ser GNU/Linux Debian 13 o Windows 11, ya que la consola se compila como un binario único sin dependencias en tiempo de ejecución. La consola publica la interfaz web en el puerto `8080` y es accedida desde un navegador instalado en el mismo equipo.

La red física funciona en modo puente (bridge) sobre la subred `192.168.1.0/24`, con un servidor DHCP/DNS en `192.168.1.1`. La tarjeta de red del anfitrión recibe `192.168.1.5` y las máquinas virtuales reciben direcciones de la misma subred en su interfaz `enp0s3`: `192.168.1.10` a `192.168.1.13`. Al estar en modo puente, las VM quedan visibles en la red local con una dirección propia, lo que permite probar servicios de red reales como SSH o HAProxy sin reglas de reenvío de puertos.

Todas las máquinas virtuales ejecutan GNU/Linux Debian 13.7.0. El balanceador ocupa un nodo propio en la fig. 1 porque es la máquina del módulo de HAProxy: recibe las peticiones en `192.168.1.11` y las reenvía, a través de la misma red, a los servidores del back-end. Las restantes se resumen en un único contenedor etiquetado como «o cualesquiera»: la VM 1 es la máquina de pruebas sobre la que se ejecutan los módulos de preparación del sistema, gestión de usuarios y servicios con systemd, y la VM 3 y la VM 4 son los servidores del back-end; la consola abre sesión con la máquina que corresponda en cada operación, de modo que el conjunto de VM cambia según las pruebas. Estas últimas se crean a partir de un *disco virtual VDI de multiconexión* (multiattach) con carga diferencial guardado en el anfitrión, de modo que cada back-end arranca desde la misma imagen base sin duplicar el disco y puede añadirse o eliminarse una máquina más utilizando el mismo archivo.

III-A2. Mecanismo de Comunicación: La consola se comunica con la máquina virtual mediante dos mecanismos, marcados como **A** y **B** en la fig. 1.

El **mecanismo A** se basa en las Guest Additions [2] y se usa *únicamente antes* de instalar SSH. La consola obtiene la dirección IP de la VM con `VBoxManage guestproperty get` sobre la propiedad `/VirtualBox/GuestInfo/Net/0/V4/IP`, genera el par de llaves en el anfitrión, instala `openssh-server`, copia la llave pública al archivo `authorized_keys` del usuario `root` y deshabilita el acceso por contraseña. Las órdenes se ejecutan con

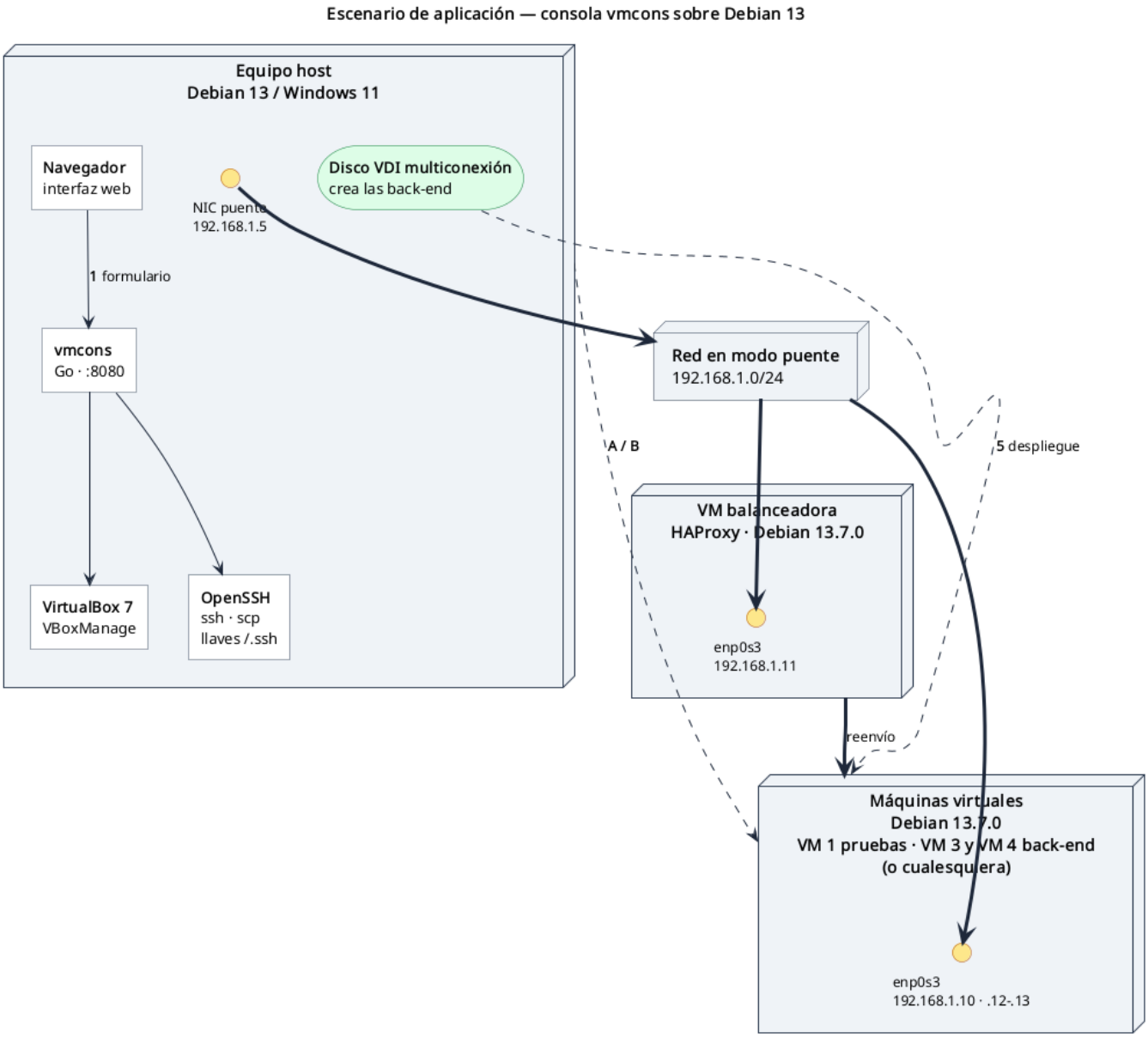


Figura 1. Escenario utilizado para desarrollar y probar la solución: equipo host, software de virtualización, máquinas virtuales con su sistema operativo, interfaces de red y direcciones IP, mecanismos de comunicación, disco de almacenamiento y flujo general de la automatización. El enlace A/B es el mecanismo de comunicación con la VM: A corresponde a las Guest Additions (solo antes de instalar SSH) y B al acceso por SSH con llaves (después, de forma exclusiva).

`VBoxManage guestcontrol run` y los archivos se transfieren con `guestcontrol copyto`.

El **mecanismo B** es el acceso por SSH con llaves de autenticación [3], mediante el cliente OpenSSH del anfitrión (`ssh` y `scp`). Una vez instalado SSH, *toda* la comunicación posterior entre la consola y la máquina se realiza por este canal, tal como exige el enunciado: el mecanismo A deja de utilizarse y la consola trabaja de forma exclusiva sobre SSH, con la llave privada resguardada en el equipo anfitrión.

III-A3. Flujo General de la Automatización: Todo módulo de la consola recorre el mismo flujo: (1) el usuario completa el formulario en el navegador y lo envía; (2) la consola valida los datos y selecciona la sesión adecuada, es decir, decide si aún necesita Guest Additions o si ya puede trabajar por SSH; (3) ejecuta la orden de forma no interactiva sobre la máquina virtual; (4) la máquina devuelve la salida y la consola notifica el resultado, o bien presenta un mensaje de error comprensible junto con la información técnica de diagnóstico; y (5) el disco multiconexión permite ampliar el escenario creando nuevas back-end para las pruebas de balanceo de carga.

III-B. Arquitectura de la Solución

III-B1. Visión General de la Aplicación: La consola es una aplicación web escrita en Go que se sirve con

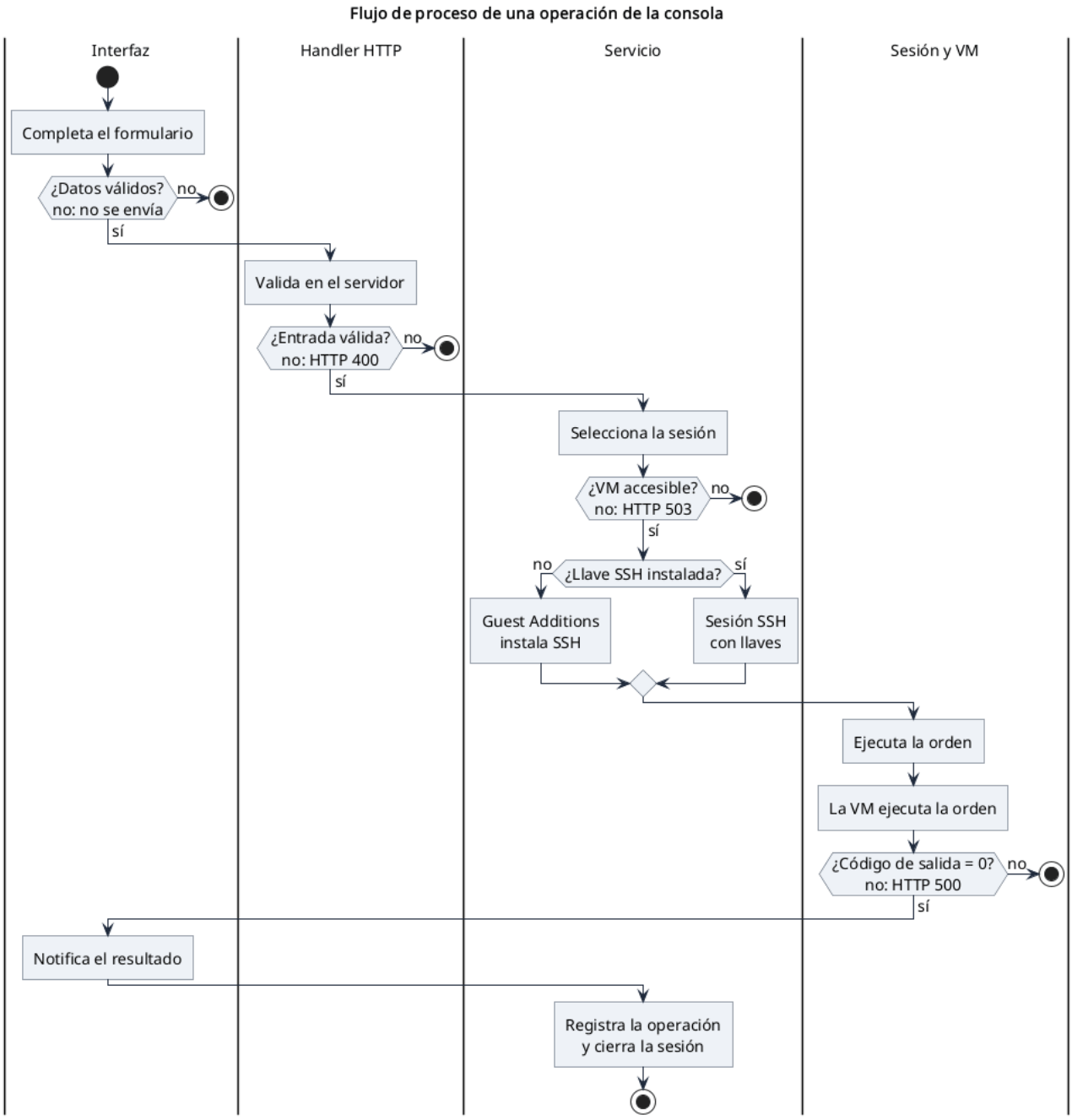


Figura 2. Flujo de proceso de una operación solicitada desde la interfaz, con los responsables de cada etapa: interfaz, validación en el servidor, selección de sesión, ejecución no interactiva y notificación del resultado o del error técnico.

el paquete estándar `net/http`. La interfaz está organizada por módulos —preparación del sistema, gestión de usuarios, servicios con systemd y balanceo de carga—, y cada módulo presenta un formulario con los campos y las casillas de verificación que la operación requiere.

Toda operación sigue el ciclo de la fig. 2: validación en el cliente para dar respuesta inmediata, validación en el servidor sobre los parámetros crudos (que es la que realmente decide), selección de la sesión, ejecución no interactiva de la orden y notificación del resultado. Si la validación falla, la operación *no* llega a la máquina virtual y la respuesta indica el campo y el motivo; si la orden se ejecuta y termina con un código de salida distinto de cero, la respuesta conserva la orden ejecutada, el código de salida y la salida de error estándar para que el usuario pueda diagnosticar el problema. Las respuestas son documentos JSON con un campo `ok`, un `mensaje` comprensible y, cuando falla, los campos `operacion` y `tecnico` con el detalle técnico.

III-B2. Estructura del Proyecto: El proyecto se organiza en capas para que el núcleo de la aplicación no dependa de VirtualBox, de las Guest Additions ni de

OpenSSH:

- `main.go`: único punto de composición. Aquí se instancian los adaptadores y se inyectan a los servicios y a los manejadores HTTP.
- `internal/core/domain`: tipos del dominio (`Command`, `Result`, `Target`, `MachineState` y `ExitError`) sin dependencias externas.
- `internal/core/ports`: interfaces `Executor`, `Machine`, `Connector` y `Session` que expresan lo que la aplicación necesita del exterior.
- `internal/adapters`: las implementaciones concretas `localproc` (procesos del sistema operativo anfitrión), `vbox` (VirtualBox y Guest Additions) y `ssh` (cliente OpenSSH).
- `internal/services`: los casos de uso de cada módulo; dependen solo de los puertos del núcleo.
- `internal/handlers`: enrutan las peticiones HTTP y validan las entradas; no contienen lógica de negocio.
- `web`: los módulos HTML y los archivos estáticos de la interfaz.

Los adaptadores traducen su vocabulario al dominio: por ejemplo, `vbox` convierte el estado `poweroff` de VirtualBox en `domain.MachinePoweredOff`, de modo que el núcleo nunca necesita saber qué tecnología se está utilizando. La regla de dependencia es simple: el núcleo no importa adaptadores, los adaptadores importan el núcleo y la composición ocurre únicamente en `main.go`.

III-B3. Decisiones Tomadas Durante el Desarrollo: Las decisiones principales del diseño son las siguientes.

Binarios del sistema, no librerías nativas. El SDK de VirtualBox exige COM en Windows y bindings específicos en GNU/Linux; en su lugar se invoca `VBoxManage`, oficial y multiplataforma, y para SSH se usa el cliente incluido en el sistema en lugar de una implementación en Go [4].

Errores tipados. Las órdenes que terminan con un código distinto de cero producen un `domain.ExitError` que conserva el código, la salida estándar y la salida de error; el núcleo lo entiende sin importar qué adaptador lo originó.

Validación antes de ejecutar y en el servidor. Las direcciones se modelan con `net/netip.Addr`, los nombres de usuario y de host se comprueban con expresiones regulares y ninguna entrada cruda llega a la máquina virtual.

Un solo flujo para los siete módulos. Todos comparten el ciclo entrada, validación, sesión, ejecución, clasificación y notificación, lo que uniformiza los mensajes de error y hace que las pruebas sean repetibles.

Calidad de código verificable. El flujo de trabajo exige formateo con `gofumpt`, análisis con `golangci-lint` y pruebas con detector de condiciones de carrera antes de cada commit.

IV. Implementación

IV-A. Preparación del Sistema

IV-A1. Instalación y Eliminación de Aplicaciones:

IV-A2. Configuración de la Llave de Acceso SSH para el Usuario Root:

IV-A3. Personalización de la Entrada del Sistema:

IV-A4. Asignación del Nombre de Host y Actualización del Archivo de Hosts:

IV-B. Gestión de Usuarios

IV-B1. Creación de Usuarios y Privilegios de sudo:

IV-B2. Eliminación de Usuarios:

IV-C. Gestión de Demonios con systemd

IV-D. Gestión de un Balanceador de Carga con HAProxy

V. Pruebas

V-A. Diseño de las Pruebas

V-B. Resultados y Evidencias

VI. Conclusiones

VI-A. Problemas Encontrados y Soluciones

VI-B. Herramientas de Inteligencia Artificial Utilizadas

VI-C. Conclusiones del Trabajo

Referencias

[1] *Oracle VM VirtualBox User Manual*, 7.1, Oracle, 2025. dirección: https://www.virtualbox.org/manual/

[2] *VirtualBox Guest Additions*, Oracle, 2025. dirección: https://www.virtualbox.org/manual/ch04.html

[3] T. Ylonen y C. Lonvick, *The Secure Shell (SSH) Authentication Protocol*, RFC 4252, Internet Engineering Task Force, 2006. dirección: https://www.rfc-editor.org/rfc/rfc4252

[4] A. A. A. Donovan y B. W. Kernighan, *The Go Programming Language*. Boston: Addison-Wesley, 2015.

[5] A. Cockburn, *Hexagonal Architecture*, 2005. dirección: https : / / alistair . cockburn . us / hexagonal - architecture